

```
In [1]: !pip install sentence-transformers scikit-learn pandas numpy
```

Requirement already satisfied: sentence-transformers in /usr/local/lib/python3.12/dist-packages (5.1.2)

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)

Requirement already satisfied: transformers<5.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.57.3)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.67.1)

Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (2.9.0+cpu)

Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (1.16.3)

Requirement already satisfied: huggingface-hub>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (0.36.0)

Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (11.3.0)

Requirement already satisfied: typing_extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transformers) (4.15.0)

Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.20.0->sentence-transformers) (3.20.0)

Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.20.0->sentence-transformers) (2025.3.0)

Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.20.0->sentence-transformers) (25.0)

Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.20.0->sentence-transformers) (6.0.3)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.20.0->sentence-transformers) (2.32.4)

Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.20.0->sentence-transformers) (1.2.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.6.1)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11.0->sentence-transformers) (3.1.6)

Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers<5.0.0,>=4.41.0->sentence-transformers) (2025.11.3)

Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers<5.0.0,>=4.41.0->sentence-transformers) (0.22.1)

Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/di

st-packages (from transformers<5.0.0,>=4.41.0->sentence-transformers) (0.7.0)
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.11.0->sentence-transformers) (1.3.0)
 Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.11.0->sentence-transformers) (3.0.3)
 Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface-hub>=0.20.0->sentence-transformers) (3.4.4)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface-hub>=0.20.0->sentence-transformers) (3.11)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface-hub>=0.20.0->sentence-transformers) (2.5.0)
 Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface-hub>=0.20.0->sentence-transformers) (2025.11.12)

```
In [82]: import numpy as np
import pandas as pd
from sentence_transformers import SentenceTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, precision_recall_curve, f1_score
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import CountVectorizer
from collections import defaultdict
```

```
In [83]: df = pd.read_csv("cosmetics_clean.csv")
df.head()
```

Out[83]:

	Label	Brand	Name	Price	Rank	Ingredients	Combination	Di
--	-------	-------	------	-------	------	-------------	-------------	----

0	Moisturizer	LA MER	Crème de la Mer	125	4.1	Algae (Seaweed) Extract, Mineral Oil, Petrolat...	1	
1	Moisturizer	SK-II	Facial Treatment Essence	125	4.1	Galactomyces Ferment Filtrate (Pitera), Butyle...	1	
2	Moisturizer	DRUNK Elephant	Protini™ Polypeptide Cream	68	4.4	Water, Dicaprylyl Carbonate, Glycerin, Ceteary...	1	
3	Moisturizer	LA MER	The Moisturizing Soft Cream	125	3.8	Algae (Seaweed) Extract, Cyclopentasiloxane, P...	1	
4	Moisturizer	IT Cosmetics	Your Skin But Better™ CC+™ Cream with SPF 50+	38	4.1	Water, Snail Secretion Filtrate, Phenyl Trimet...	1	

```
In [133... text_cols = ["Label", "Brand", "Name", "Ingredients"]
skin_cols = ["Combination", "Dry", "Normal", "Oily", "Sensitive"]
num_cols = ["Price", "Rank"]
```

```
In [134... def build_text(row):
    parts = []
    for col in text_cols:
        if pd.notna(row[col]):
            parts.append(str(row[col]))
    return " || ".join(parts)

df["text"] = df.apply(build_text, axis=1)
texts = df["text"].tolist()
```

```
In [135... sbert = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = sbert.encode(
    texts,
    batch_size=64,
    convert_to_numpy=True,
    normalize_embeddings=True
)
```

```
In [136... embeddings.shape
```

```
Out[136... (1472, 384)
```

```
In [137... numX = df[num_cols].fillna(df[num_cols].median()).to_numpy()
scaler = StandardScaler()
numX = scaler.fit_transform(numX)

X = np.hstack([embeddings, numX])
Y = df[skin_cols].astype(int).to_numpy()
```

```
In [138... X.shape, Y.shape
```

```
Out[138... ((1472, 386), (1472, 5))
```

```
In [139... X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)

base_models = {}
base_thresholds = {}
```

```
In [140... def find_best_threshold(y_true, y_prob):
    precision, recall, thresholds = precision_recall_curve(y_true, y_prob)
    f1s = 2 * precision * recall / (precision + recall + 1e-9)
    idx = np.argmax(f1s)
    return thresholds[idx-1] if idx > 0 else 0.5
```

```
In [141... for i, skin in enumerate(skin_cols):
    clf = LogisticRegression(max_iter=500, class_weight="balanced")
    clf.fit(X_train, Y_train[:, i])

    y_prob = clf.predict_proba(X_train)[:, 1]
    thr = find_best_threshold(Y_train[:, i], y_prob)
```

```
base_models[skin] = clf
base_thresholds[skin] = thr
```

```
In [142... sim_matrix = cosine_similarity(embeddings, embeddings)
```

```
In [143... def get_neighbors(idx, k=10):
    sims = sim_matrix[idx]
    top_idx = np.argsort(-sims)[1:k+1]
    return top_idx, sims[top_idx]
```

```
In [144... ingredient_map = {skin: {} for skin in skin_cols}

for skin in skin_cols:
    skin_df = df[df[skin] == 1]
    for ing_list in skin_df["Ingredients"].dropna():
        for ing in ing_list.lower().split(","):
            ing = ing.strip()
            ingredient_map[skin][ing] = ingredient_map[skin].get(ing, 0) + 1
```

```
In [145... def ingredient_score(ingredients, skin):
    if pd.isna(ingredients):
        return 0.0
    score = 0
    for ing in ingredients.lower().split(","):
        score += ingredient_map[skin].get(ing.strip(), 0)
    return score
```

```
In [146... def adaptive_threshold(base_thr, similarity_scores):
    density = np.mean(similarity_scores > 0.7)
    if density > 0.3:
        return base_thr - 0.05
    else:
        return base_thr + 0.05
```

```
In [147... def refine_with_similarity(prob, neighbor_labels):
    agreement = np.mean(neighbor_labels)
    return 0.7 * prob + 0.3 * agreement
```

```
In [148... def cold_start_handler(probs, neighbor_labels):
    if max(probs.values()) < 0.55:
        return {k: np.mean(neighbor_labels[k]) for k in probs}
    return probs
```

```
In [149... ingredient_skin_graph = defaultdict(lambda: defaultdict(int))

for _, row in df.iterrows():
    if pd.isna(row["Ingredients"]):
        continue
    ingredients = [i.strip().lower() for i in row["Ingredients"].split(",")]
    for skin in skin_cols:
        if row[skin] == 1:
            for ing in ingredients:
                ingredient_skin_graph[ing][skin] += 1
```

```
In [150... ingredient_skin_prob = {}

for ing, skin_counts in ingredient_skin_graph.items():
```

```
total = sum(skin_counts.values())
ingredient_skin_prob[ing] = {
    skin: count / total for skin, count in skin_counts.items()
}
```

```
In [151... def graph_skin_score(ingredients, skin):
    if pd.isna(ingredients):
        return 0.0
    score = 0
    cnt = 0
    for ing in ingredients.lower().split(","):
        ing = ing.strip()
        if ing in ingredient_skin_prob and skin in ingredient_skin_prob[ing]:
            score += ingredient_skin_prob[ing][skin]
            cnt += 1
    return score / cnt if cnt > 0 else 0.0
```

```
In [152... def graph_ingredient_score(ingredients, skin):
    if pd.isna(ingredients):
        return 0.0

    score = 0.0
    count = 0

    for ing in ingredients.lower().split(","):
        ing = ing.strip()
        if ing in ingredient_skin_prob:
            if skin in ingredient_skin_prob[ing]:
                score += ingredient_skin_prob[ing][skin]
                count += 1

    if count == 0:
        return 0.0

    return score / count
```

```
In [153... def predict_product(idx, MAX_LABELS=3):
    neighbors, sim_scores = get_neighbors(idx)

    raw_probs = {}

    for i, skin in enumerate(skin_cols):
        clf = base_models[skin]

        base_prob = clf.predict_proba(
            X[idx].reshape(1, -1)
        )[0, 1]

        neighbor_labels = Y[neighbors, i]
        prob = refine_with_similarity(base_prob, neighbor_labels)

        # Graph-based ingredient reasoning
        graph_score = graph_ingredient_score(
            df.iloc[idx]["Ingredients"], skin
        )
        prob = 0.75 * prob + 0.25 * graph_score

    prob = float(np.clip(prob, 0.0, 1.0))
```

```

        raw_probs[skin] = prob

    raw_probs = cold_start_handler(
        raw_probs,
        {skin: Y[neighbors, i] for i, skin in enumerate(skin_cols)}
    )

    final_probs = {}
    final_preds = {}

    for skin in skin_cols:
        thr = adaptive_threshold(base_thresholds[skin], sim_scores)
        prob = raw_probs[skin]

        final_probs[skin] = prob
        final_preds[skin] = int(prob >= thr)

    top_skins = sorted(
        final_probs.items(),
        key=lambda x: x[1],
        reverse=True
    )

    allowed = set([k for k, _ in top_skins[:MAX_LABELS]])

    for skin in final_preds:
        final_preds[skin] = int(
            skin in allowed and final_preds[skin] == 1
        )

    return final_preds, final_probs

```

In [154...

```

idx = 264
preds, probs = predict_product(idx)

print("Product:", df.iloc[idx]["Name"])
print("Predicted suitability:")
for k in preds:
    print(k, preds[k], f"(prob={probs[k]:.2f})")

```

```

Product: Moon Fruit Superfruit Night Treatment
Predicted suitability:
Combination 0 (prob=0.51)
Dry 1 (prob=0.55)
Normal 1 (prob=0.52)
Oily 0 (prob=0.51)
Sensitive 1 (prob=0.54)

```

```
In [155... def explain_similarity(idx, skin, top_k=3):
    neighbors, sim_scores = get_neighbors(idx, k=top_k)
    explanations = []

    for n_idx, sim in zip(neighbors, sim_scores):
        explanations.append({
            "product": df.iloc[n_idx]["Name"],
            "similarity": round(float(sim), 3),
            "skin_label": int(df.iloc[n_idx][skin])
        })

    return explanations
```

```
In [156... def explain_ingredients(ingredients, skin, top_k=3):
    if pd.isna(ingredients):
        return []

    contributions = []

    for ing in ingredients.lower().split(","):
        ing = ing.strip()
        if ing in ingredient_skin_prob and skin in ingredient_skin_prob[ing]:
            contributions.append((
                ing,
                ingredient_skin_prob[ing][skin]
            ))

    contributions.sort(key=lambda x: x[1], reverse=True)

    return [
        {"ingredient": ing, "support": round(score, 3)}
        for ing, score in contributions[:top_k]
    ]
```

```
In [157... def explain_decision_control(
    skin,
    prob,
    thr,
    allowed,
    graph_score=None,
    sim_support=None
):
    reasons = []

    if prob >= thr:
        reasons.append(
            f"Probability {prob:.2f} exceeded threshold {thr:.2f}"
        )
    else:
        reasons.append(
            f"Probability {prob:.2f} was below threshold {thr:.2f}"
        )

    if skin not in allowed:
        reasons.append(
            "Lower priority compared to other skin types"
        )
    else:
        reasons.append(
```



```

        "Ranked among top skin types for this product"
    )

    if graph_score is not None:
        if graph_score > 0.5:
            reasons.append(
                "Ingredients strongly associated with this skin type"
            )
        elif graph_score > 0:
            reasons.append(
                "Ingredients moderately associated with this skin type"
            )
        else:
            reasons.append(
                "Ingredients provide little direct support for this skin type"
            )

    if sim_support is not None:
        if sim_support > 0.6:
            reasons.append(
                "Similar products frequently support this skin type"
            )
        elif sim_support > 0:
            reasons.append(
                "Mixed evidence from similar products"
            )
        else:
            reasons.append(
                "Similar products rarely support this skin type"
            )

    return reasons

```

In [158...

```

def explain_prediction(idx, MAX_LABELS=3):
    preds, probs = predict_product(idx, MAX_LABELS)
    neighbors, sim_scores = get_neighbors(idx)

    explanations = {}

    sorted_probs = sorted(probs.items(), key=lambda x: x[1], reverse=True)
    allowed = set([k for k, _ in sorted_probs[:MAX_LABELS]])

    for skin in skin_cols:
        thr = adaptive_threshold(base_thresholds[skin], sim_scores)

        skin_idx = skin_cols.index(skin)
        neighbor_labels = Y[neighbors, skin_idx]
        sim_support = (
            float(neighbor_labels.mean()) if len(neighbor_labels) > 0 else 0.0
        )
        graph_score = graph_ingredient_score(
            df.iloc[idx]["Ingredients"], skin
        )

        explanations[skin] = {
            "final_prediction": preds[skin],
            "final_probability": round(float(probs[skin]), 3),
            "adaptive_threshold": round(float(thr), 3),

            # evidence

```

```

        "similar_products": explain_similarity(idx, skin),
        "ingredient_evidence": explain_ingredients(
            df.iloc[idx]["Ingredients"], skin
        ),

        # enriched decision reasoning
        "decision_control": explain_decision_control(
            skin=skin,
            prob=probs[skin],
            thr=thr,
            allowed=allowed,
            graph_score=graph_score,
            sim_support=sim_support
        )
    }

    return explanations

```

In [159...

```

idx = 264
explanations = explain_prediction(idx)

print("Product:", df.iloc[idx]["Name"])
print()

for skin, info in explanations.items():
    print(f"--- {skin} ---")
    print("Prediction:", info["final_prediction"])
    print("Probability:", info["final_probability"])
    print("Threshold:", info["adaptive_threshold"])
    print("Decision control:", info["decision_control"])

    print("Top similar products:")
    for sp in info["similar_products"]:
        print(" ", sp)

    print("Ingredient evidence:")
    for ing in info["ingredient_evidence"]:
        print(" ", ing)

    print()

```

Product: Moon Fruit Superfruit Night Treatment

--- Combination ---

Prediction: 0

Probability: 0.511

Threshold: 0.324

Decision control: ['Probability 0.51 exceeded threshold 0.32', 'Lower priority compared to other skin types', 'Ingredients moderately associated with this skin type', 'Similar products frequently support this skin type']

Top similar products:

```
{'product': 'Exfoliating Peel Gel', 'similarity': 0.889, 'skin_label': 1}
{'product': 'Beauty Water', 'similarity': 0.886, 'skin_label': 1}
{'product': 'Hyaluronic Marine Oil-Free Moisture Cushion', 'similarity': 0.885, 'skin_label': 1}
```

Ingredient evidence:

```
{'ingredient': 'glyceryl stearate', 'support': 0.223}
{'ingredient': 'xanthan gum', 'support': 0.217}
{'ingredient': 'hyaluronic acid', 'support': 0.217}
```

--- Dry ---

Prediction: 1

Probability: 0.551

Threshold: 0.386

Decision control: ['Probability 0.55 exceeded threshold 0.39', 'Ranked among top skin types for this product', 'Ingredients moderately associated with this skin type', 'Similar products frequently support this skin type']

Top similar products:

```
{'product': 'Exfoliating Peel Gel', 'similarity': 0.889, 'skin_label': 1}
{'product': 'Beauty Water', 'similarity': 0.886, 'skin_label': 1}
{'product': 'Hyaluronic Marine Oil-Free Moisture Cushion', 'similarity': 0.885, 'skin_label': 1}
```

Ingredient evidence:

```
{'ingredient': 'hyaluronic acid', 'support': 0.217}
{'ingredient': 'cetearyl alcohol', 'support': 0.212}
{'ingredient': 'glyceryl stearate', 'support': 0.205}
```

--- Normal ---

Prediction: 1

Probability: 0.523

Threshold: 0.381

Decision control: ['Probability 0.52 exceeded threshold 0.38', 'Ranked among top skin types for this product', 'Ingredients moderately associated with this skin type', 'Similar products frequently support this skin type']

Top similar products:

```
{'product': 'Exfoliating Peel Gel', 'similarity': 0.889, 'skin_label': 1}
{'product': 'Beauty Water', 'similarity': 0.886, 'skin_label': 1}
{'product': 'Hyaluronic Marine Oil-Free Moisture Cushion', 'similarity': 0.885, 'skin_label': 1}
```

Ingredient evidence:

```
{'ingredient': 'glyceryl stearate', 'support': 0.22}
{'ingredient': 'cetearyl alcohol', 'support': 0.22}
{'ingredient': 'hyaluronic acid', 'support': 0.217}
```

--- Oily ---

Prediction: 0

Probability: 0.508

Threshold: 0.295

Decision control: ['Probability 0.51 exceeded threshold 0.30', 'Lower priority compared to other skin types', 'Ingredients moderately associated with this skin type', 'Similar products frequently support this skin type']

```

Top similar products:
  {'product': 'Exfoliating Peel Gel', 'similarity': 0.889, 'skin_label': 1}
  {'product': 'Beauty Water', 'similarity': 0.886, 'skin_label': 1}
  {'product': 'Hyaluronic Marine Oil-Free Moisture Cushion', 'similarity': 0.885,
'skin_label': 1}
Ingredient evidence:
  {'ingredient': 'xanthan gum', 'support': 0.205}
  {'ingredient': 'organic aloe leaf juice (aloe barbadensis)', 'support': 0.2}
  {'ingredient': 'red raspberry seed oil (rubus idaeus)', 'support': 0.2}

--- Sensitive ---
Prediction: 1
Probability: 0.541
Threshold: 0.353
Decision control: ['Probability 0.54 exceeded threshold 0.35', 'Ranked among top
skin types for this product', 'Ingredients moderately associated with this skin t
ype', 'Similar products frequently support this skin type']
Top similar products:
  {'product': 'Exfoliating Peel Gel', 'similarity': 0.889, 'skin_label': 1}
  {'product': 'Beauty Water', 'similarity': 0.886, 'skin_label': 1}
  {'product': 'Hyaluronic Marine Oil-Free Moisture Cushion', 'similarity': 0.885,
'skin_label': 1}
Ingredient evidence:
  {'ingredient': 'organic aloe leaf juice (aloe barbadensis)', 'support': 0.2}
  {'ingredient': 'red raspberry seed oil (rubus idaeus)', 'support': 0.2}
  {'ingredient': 'organic prickly pear oil (opuntia ficus-Indica)', 'support': 0.
2}

```

```

In [160... user_input = {
    "product_types": ["Face Mask"],
    "skin_types": ["Oily"],
    "skin_concerns": ["very oily skin"],
    "avoid_ingredients": [],
    "preferred_ingredients": ["charcoal powder", "peat", "lava powder"],
    "priority": "gentle",
    "reference_product": None,
    "top_k": 5
}

```

```

In [161... def contains_avoided_ingredients(product_ingredients, avoid_list):
    if pd.isna(product_ingredients):
        return False

    ingredients = product_ingredients.lower()
    for bad in avoid_list:
        if bad.lower() in ingredients:
            return True
    return False

```

```

In [162... def matches_product_type(product_label, allowed_types):
    if not allowed_types:
        return True
    return product_label in allowed_types

```

```

In [163... def preferred_ingredient_score(ingredients, preferred):
    if not preferred or pd.isna(ingredients):
        return 0.0

```

```

ingredients = ingredients.lower()
return sum(1 for p in preferred if p.lower() in ingredients) / len(preferred

```

In [164...

```

def skin_concern_score(ingredients, concerns):
    if not concerns or pd.isna(ingredients):
        return 0.0

    ingredients = ingredients.lower()
    score = 0

    for c in concerns:
        if c == "acne" and "salicylic" in ingredients:
            score += 1
        if c == "dryness" and ("glycerin" in ingredients or "hyaluronic" in ingr
            score += 1
        if c == "aging" and ("retinol" in ingredients or "peptide" in ingredient
            score += 1

    return score / max(len(concerns), 1)

```

In [165...

```

def recommend_products(user_input):
    results = []

    for idx in range(len(df)):
        if not matches_product_type(
            df.iloc[idx]["Label"],
            user_input.get("product_types", [])
        ):
            continue

        preds, probs = predict_product(idx)

        if not any(preds[s] == 1 for s in user_input["skin_types"]):
            continue

        if contains_avoided_ingredients(
            df.iloc[idx]["Ingredients"],
            user_input.get("avoid_ingredients", [])
        ):
            continue

        base_score = max(probs[s] for s in user_input["skin_types"])
        concern_score = skin_concern_score(
            df.iloc[idx]["Ingredients"],
            user_input.get("skin_concerns", [])
        )

        pref_score = preferred_ingredient_score(
            df.iloc[idx]["Ingredients"],
            user_input.get("preferred_ingredients", [])
        )

        final_score = (
            0.6 * base_score +
            0.2 * concern_score +
            0.2 * pref_score
        )

        results.append({
            "index": idx,

```

```

        "name": df.iloc[idx]["Name"],
        "brand": df.iloc[idx]["Brand"],
        "label": df.iloc[idx]["Label"],
        "score": round(final_score, 3)
    })

results.sort(key=lambda x: x["score"], reverse=True)
return results[: user_input.get("top_k", 5)]

```

In [166...

```

results = recommend_products(user_input)

for r in results:
    print("=" * 50)
    print("Product:", r["name"])
    print("Brand:", r["brand"])
    print("Type:", r["label"])
    print("Score:", r["score"])

    explanations = explain_prediction(r["index"])

    for skin in user_input["skin_types"]:
        info = explanations[skin]
        print(f"    [{skin}] → {info['final_prediction']} (prob={info['final_proba']})
        print("    Reason:", info["decision_control"])

```

```

=====
Product: Clarifying Colloidal Sulfur Mask
Brand: DR. DENNIS GROSS Skincare
Type: Face Mask
Score: 0.54
[Oily] → 1 (prob=0.9)
Reason: ['Probability 0.90 exceeded threshold 0.40', 'Ranked among top skin ty
pes for this product', 'Ingredients provide little direct support for this skin t
ype', 'Similar products frequently support this skin type']
=====
Product: Flower Fusion™ Rose Hydrating Sheet Mask
Brand: Origins
Type: Face Mask
Score: 0.54
[Oily] → 1 (prob=0.9)
Reason: ['Probability 0.90 exceeded threshold 0.30', 'Ranked among top skin ty
pes for this product', 'Ingredients moderately associated with this skin type',
'Similar products frequently support this skin type']
=====
Product: Irish Moor Mud Purifying Black Mask
Brand: PETER THOMAS ROTH
Type: Face Mask
Score: 0.537
[Oily] → 1 (prob=0.562)
Reason: ['Probability 0.56 exceeded threshold 0.30', 'Ranked among top skin ty
pes for this product', 'Ingredients moderately associated with this skin type',
'Similar products frequently support this skin type']
=====
Product: Instant Detox Hawaiian Sea Water & White Clay Face Mask
Brand: BOBBI BROWN
Type: Face Mask
Score: 0.487
[Oily] → 1 (prob=0.7)
Reason: ['Probability 0.70 exceeded threshold 0.30', 'Ranked among top skin ty
pes for this product', 'Ingredients moderately associated with this skin type',
'Similar products frequently support this skin type']
=====
Product: Out of Trouble™ 10 Minute Mask to Rescue Problem Skin
Brand: Origins
Type: Face Mask
Score: 0.48
[Oily] → 1 (prob=0.8)
Reason: ['Probability 0.80 exceeded threshold 0.30', 'Ranked among top skin ty
pes for this product', 'Ingredients moderately associated with this skin type',
'Similar products frequently support this skin type']

```

In [167...

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

```

In [168...

```

evaluation_results = {}

for i, skin in enumerate(skin_cols):
    clf = base_models[skin]
    y_prob = clf.predict_proba(X_test)[: , 1]
    y_pred = (y_prob >= 0.5).astype(int)

```

```
y_true = Y_test[:, i]

evaluation_results[skin] = {
    "accuracy": accuracy_score(y_true, y_pred),
    "precision": precision_score(y_true, y_pred, zero_division=0),
    "recall": recall_score(y_true, y_pred, zero_division=0),
    "f1": f1_score(y_true, y_pred, zero_division=0),
    "roc_auc": roc_auc_score(y_true, y_prob)
}
```

```
In [169... print("Evaluation metrics per skin type:\n")

for skin, metrics in evaluation_results.items():
    print(f"--- {skin} ---")
    for k, v in metrics.items():
        print(f"{k:10s}: {v:.3f}")
    print()
```

Evaluation metrics per skin type:

--- Combination ---

accuracy : 0.647
precision : 0.723
recall : 0.730
f1 : 0.726
roc_auc : 0.682

--- Dry ---

accuracy : 0.661
precision : 0.711
recall : 0.711
f1 : 0.711
roc_auc : 0.698

--- Normal ---

accuracy : 0.675
precision : 0.758
recall : 0.726
f1 : 0.742
roc_auc : 0.722

--- Oily ---

accuracy : 0.600
precision : 0.659
recall : 0.667
f1 : 0.663
roc_auc : 0.633

--- Sensitive ---

accuracy : 0.566
precision : 0.567
recall : 0.620
f1 : 0.592
roc_auc : 0.617

In [170...